COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	07/17/26	**DATE SUBMITTED:**	07/17/26
NAME:	Francis Elijah J. Formalejo	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. _2.1_ Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

What procedures did you perform?

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;
    int *data;
    size_t size;

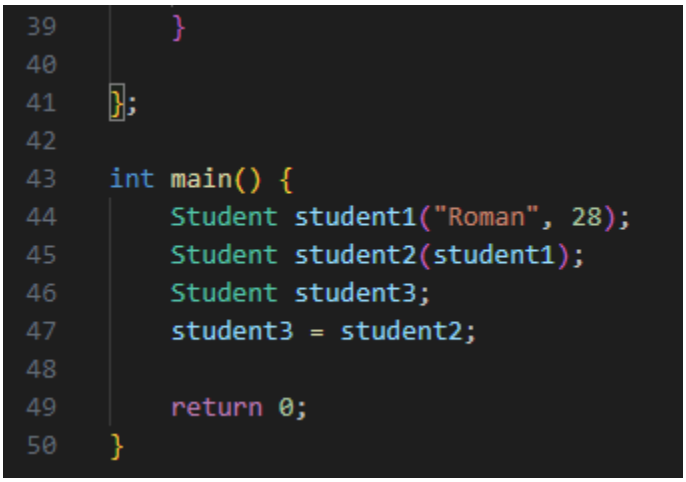
public:
    Student (std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move (newName);
        studentAge = newAge;
        std::cout << "Constructor Successfully Called." << std::endl;
    };

    ~Student () {
        std::cout << "Destructor Successfully Called." << std::endl;
    }

    Student (const Student &copyStudent) {
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
        std::cout << "Copy Constructor Successfully Called." << std::endl;
    }

    Student& operator = (const Student &copyStudent) {
        if (this != &copyStudent) {
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
            std::cout << "Copy Assignment Operator Successfully Called." << std::endl;
        }
        return *this;
    }

    void printDetails () {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
```



```
39      }
40
41  };
42
43  int main() {
44      Student student1("Roman", 28);
45      Student student2(student1);
46      Student student3;
47      student3 = student2;
48
49      return 0;
50  }
```

Observation	The program first creates student 1 that is connected to the main constructor, then creates student 2 and is connected to the copy constructor, then student 3 as our john doe that will use the copy assignment to overwrite the defualkt info with student 2, then it will delete as in ends.

Table 2-1. Initial Driver Program

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;
    int *data;
    size_t size;

public:
    Student (std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move (newName);
        studentAge = newAge;
        std::cout << "Constructor Successfully Called." << std::endl;
    };

    ~Student () {
        std::cout << "Destructor Successfully Called." << std::endl;
    }

    Student (const Student &copyStudent) {
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
        std::cout << "Copy Constructor Successfully Called." << std::endl;
    }

    Student& operator = (const Student &copyStudent) {
        if (this != &copyStudent) {
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
            std::cout << "Copy Assignment Operator Successfully Called." << std::endl;
        }
        return *this;
    }

    void printDetails () {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }

};

int main() {
    const size_t j = 5;

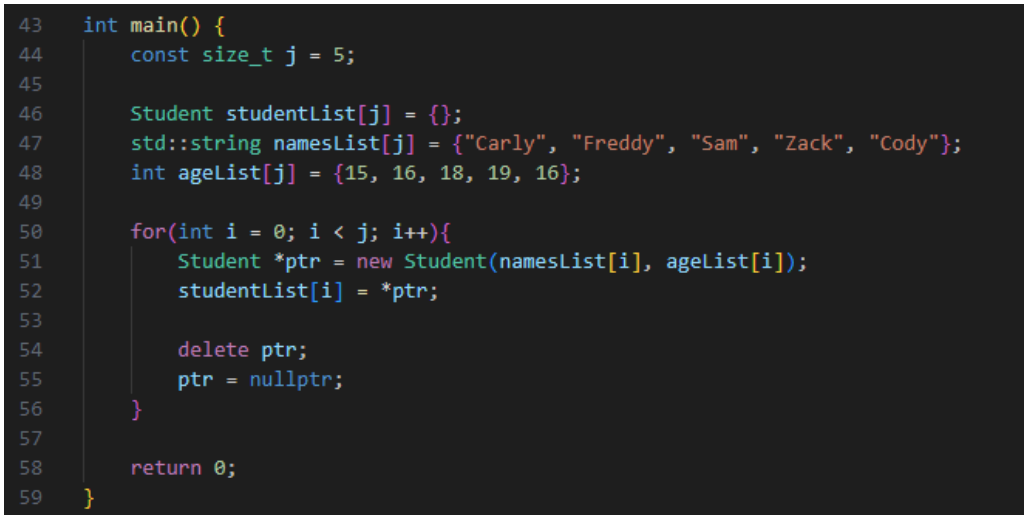
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};
```

```cpp
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

Observation	The program will blank 5 spots first like reserving it, then it will fill it with the john does and execute 5 times. Then the code will input the names and ages like carly and freddy with their ages. Then it will again delete it as it ends.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	The program will loop 5 imes and in each loop, it will create an object and stay in on the heap and then copies in to the studentlist, then the delete ptr will delete the temporary object.
Loop B	

```
43  int main() {
44      const size_t j = 5;
45
46      Student studentList[j] = {};
47      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
48      int ageList[j] = {15, 16, 18, 19, 16};
49
50      for(int i = 0; i < j; i++){
51          Student *ptr = new Student(namesList[i], ageList[i]);
52          studentList[i] = *ptr;
53
54          delete ptr;
55          ptr = nullptr;
56      }
57
58      return 0;
59  }
```

```cpp
int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++) {
        studentList[i].printDetails();
    }

    for(int i = 0; i < j; i++){
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;

        delete ptr;
        ptr = nullptr;
    }

    for(int i = 0; i < j; i++) {
        studentList[i].printDetails();
    }

    return 0;
}
```

Observation	Loop b will run through them all and print them instantly as it goes on. It's like loop a but its not that complicated
Output	Constructor Successfully Called. Copy Constructor Successfully Called. Constructor Successfully Called. Copy Assignment Operator Successfully Called. Destructor Successfully Called. Destructor Successfully Called. Destructor Successfully Called.

```
Constructor Successfully Called.
Constructor Successfully Called.
Constructor Successfully Called.
Constructor Successfully Called.
Constructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
```

```
Constructor Successfully Called.
Constructor Successfully Called.
Constructor Successfully Called.
Constructor Successfully Called.
Constructor Successfully Called.
Constructor Successfully Called.
Copy Assignment Operator Successfully Called.
Destructor Successfully Called.
Constructor Successfully Called.
Copy Assignment Operator Successfully Called.
Destructor Successfully Called.
Constructor Successfully Called.
Copy Assignment Operator Successfully Called.
Destructor Successfully Called.
Constructor Successfully Called.
Copy Assignment Operator Successfully Called.
Destructor Successfully Called.
Constructor Successfully Called.
Copy Assignment Operator Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
```

Loop b:

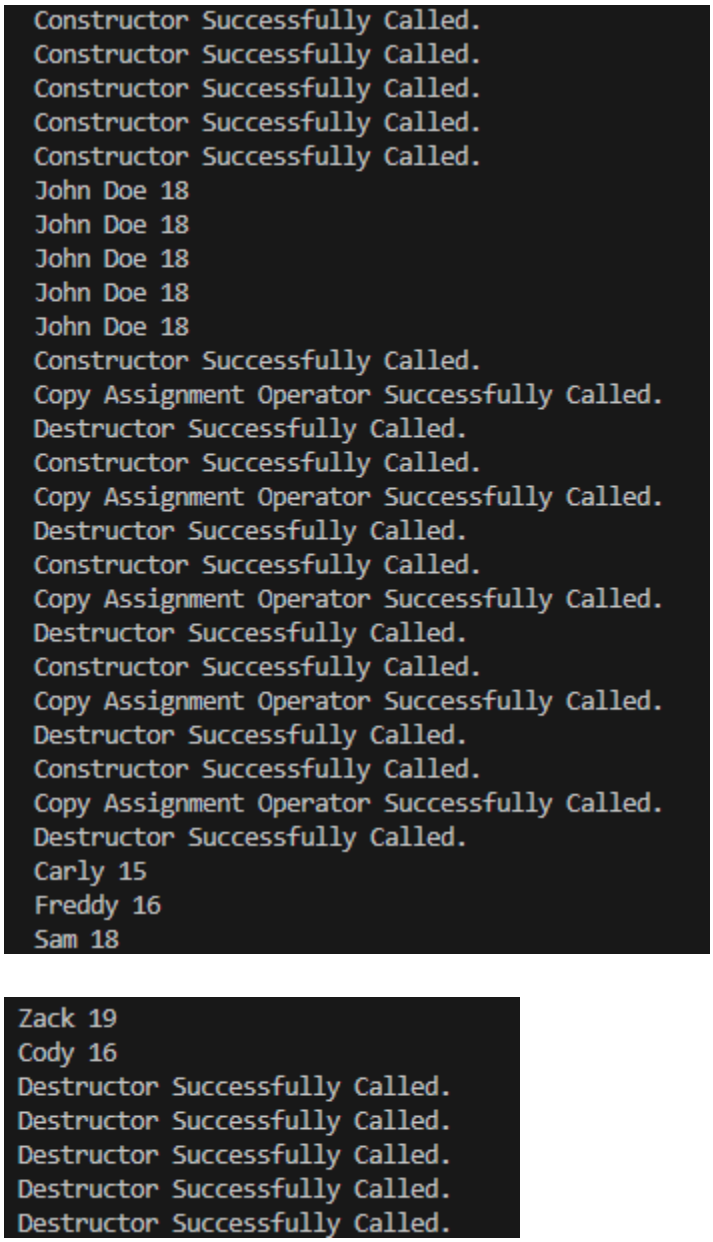
	
Observation	

Table 2-3. Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Language translation from programming language to english**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used them to translate my errors from programming language to english lol**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion:
Overall, we explored how c++ manages the memory by observing how objects are created, copied, and destroyed using both stack and heap. First we started by tracking basic objects and saw how the program initially fills a slot with default "John Doe" on the stack before using Loop A to build new objects on the heap and copy their real names over, followed by Loop B to print out the final list, then it will then delete in the end using the delete. I think I did 6/10 here since it took me a longtime to follow the instructions but overall, I kind of got bored in this one but its one of our important lessons.